

P1907R1: Inconsistencies with non-type template parameters

Introduction

Non-type template parameters were originally limited to having scalar non-floating-point types. Through a number of recent changes, non-type template parameters of class type are now supported as well, provided they have strong structural equality, in particular no user-defined operator== for any of the class's subobjects.

- [P0732R2](#) Class Types in Non-Type Template Parameters
- [P1185R2](#) <=> != ==
- [P1630R1](#) Spaceship needs a tune-up, Addressing some discovered issues with P0515 and P1185

[P1714R1](#) (NTTP are incomplete without float, double, and long double!), which added floating-point types as permissible non-type template parameters (with bit-wise comparison on the value representation), was rejected by plenary straw poll in Cologne.

The following wording implements guidance by EWG in Belfast to address the following NB comments:

- US092 11.10.01 [class.compare.default] p04.2.1 Array members should have strong structural equality
- US093 11.10.01 p4 Move definition of "strong structural equality" near its use in
- (rejected) US094 11.10.01 p4 Deleted operator== should prevent strong structural equality
- US100 13.01 p4.1 Reference types should not have strong structural equality
- US102 13.1 p4.1 Allow non-type template parameters of floating-point type P1714
- US114 13.05 p1.5 Class types as non-type template arguments

Changes

- P1907R0: Presentations of inconsistencies (not repeated here)

Wording

Remove 11.11.1 [class.compare.default] paragraph 4:

~~A type C has strong structural equality if, given a glvalue x of type const C, either:~~

~~C is a non-class type and x <=> x is a valid expression of type std::strong_ordering or std::strong_equality, or~~

- ~~C is a class type where all of the following hold:~~
 - ~~All of C's base class subobjects and non-static data members have strong structural equality.~~
 - ~~C has no mutable or volatile non-static data members.~~
 - ~~At the end of the definition of C, overload resolution performed for the expression x == x succeeds and finds either a friend or public member == operator that is defined as defaulted in the definition of C.~~

Change in 13.2 [temp.param] paragraph 4:

A non-type *template-parameter* shall have one of the following (optionally cv-qualified) types:

- ~~a literal type that has strong structural equality (11.11.1);~~
- ~~an lvalue reference type;~~
- **a structural type (see below),**
- a type that contains a placeholder type (9.2.8.5), or
- a placeholder for a deduced class type (9.2.8.6).

~~[Note: Other types are disallowed either explicitly below or implicitly by the rules governing the form of template arguments (13.4 [temp.arg]). — end note]~~ The top-level cv-qualifiers on the *template-parameter* are ignored when determining its type.

Change in 13.4.2 [temp.arg.nontype] paragraph 2:

A template-argument for a non-type template-parameter shall be a converted constant expression (7.7) of the type of the template-parameter. For a non-type template-parameter of reference or pointer type, or for each non-static data member of reference or pointer type in a non-type template-parameter of class type or subobject thereof, the reference or pointer value shall not refer to or be the address of (respectively):

- ~~a subobject (6.7.2);~~
- a temporary object (6.7.7),
- a string literal (5.13.5),
- the result of a typeid expression (7.6.1.7), ~~or~~
- a predefined `__func__` variable (9.5.1); ~~or~~
- **a subobject (6.7.2) of one of the above.**

A structural type is one of the following:

- **a scalar type, or**
- **an lvalue reference type, or**
- **a literal class type with the following properties:**
 - **all base classes and non-static data members are public and non-mutable and**
 - **the types of all bases classes and non-static data members are structural types or (possibly multi-dimensional) array thereof.**

Change in 13.2 [temp.param] paragraph 6:

An *id-expression* naming a non-type template-parameter of class type T denotes a static storage duration object of type const T, known as a *template parameter object*, whose value is that of the corresponding template argument after it has been converted to the type of the template-parameter. All such template parameters in the program of the same type with the same value denote the same template parameter object. **A template parameter object shall have constant destruction (7.7 [expr.const]).** [Note: If an id-expression names a non-type non-reference template-parameter, then it is a prvalue if it has non-class type. Otherwise, if it is of class type T, it is an lvalue and has type const T (7.5.4.1). — end note] [Example:

Change in 13.6 [temp.type] paragraph 1:

Two *template-ids* refer to the same class, function, or variable if

- their *template-names*, *operator-function-ids*, or *literal-operator-ids* refer to the same template and
- their corresponding type *template-arguments* are the same type and

- their corresponding non-type *template-arguments* of pointer-to-member type refer to the same class member or are both the null member pointer value and
- their corresponding non-type *template-arguments* of reference type refer to the same object or function and
- their remaining corresponding non-type *template-arguments* have the same type and value **are template-argument-equivalent (see below)** after conversion to the type of the *template-parameter*, where they are considered to have the same value if they compare equal with the `==` operator (7.6.10), and
- their corresponding template *template-arguments* refer to the same template.

Two values are *template-argument-equivalent* if they are of the same type and

- they are of integral type and their values are the same, or
- they are of floating-point type and their values are identical, or
- they are of type `std::nullptr_t`, or
- they are of enumeration type τ and their values are the same [Footnote: The identity of enumerators is not preserved.], or
- they are of pointer type and they have the same pointer value, or
- they are of pointer-to-member type and they refer to the same class member or are both the null member pointer value, or
- they are of reference type and they refer to the same object or function, or
- they are of array type and their corresponding elements are template-argument-equivalent[Footnote: An array as a *template-parameter* decays to a pointer.], or
- they are of union type and either they both have no active member or they have the same active member and their active members are template-argument-equivalent, or
- they are of class type and their corresponding direct subobjects and reference members are template-argument-equivalent.